

```
# Common NLP Tasks
# - sentiment analysis
# - conversational agents
# - knowledge graph and QA systems
# - summarization
# - topic modelling
# - typing behaviour
# - text parsing into noun verb
# - speech to text
```

```
# Approaches to NLP
# - Heuristic methods -
# -- regular expressions, wordnet, open mind common sense
# - machine learning methods
# - deep learning methods
# -- deep learning algorithms retain data in sequential order
# -- auto feature selection
```

```
#text augmentation methods
#replace words with its synonym
#bigram flip
```

```
#nlp pipeline
#1.data acquisition
#2.data preprocessing
#3.feature engineering
#4.modelling
#5.deploy
```

✓ Text Preprocessing

```
import pandas as pd
import re
import string
import emoji
import spacy
from textblob import TextBlob
from nltk.corpus import stopwords
# def remove_htmltag(text):
#     pattern=re.compile('<.*?>')
#     return pattern.sub(r'',text)

# def remove_urls(text):
#     pattern=re.compile(r'https?://S+|www.\.\S+')
#     return pattern.sub(r'',text)

# exclude=string.punctuation
# def remove_punc(text):
#     for char in exclude:
#         text=text.replace(char,'')
#     return text
# def correct_spellings(text):
#     textinc=TextBlob(text)
#     text=textinc.correct().string
#     return text
# def remove_emoji(text):
#     emoji_pattern=re.compile(
#     # '['
#     #     u'\U0001F600-\U0001F64F'
#     #     u'\U0001F600-\U0001F64F'
#     #     u'\U0001F300-\U0001F5FF'
#     #     u'\U0001F680-\U0001F6FF'
#     #     u'\U0001F680-\U0001F6FF'
#     #     ']'
#     #     flags=re.UNICODE
#     # )
#     return emoji_pattern.sub(r'',text)

# def remove_stopwords(text):
#     new_text=[]
#     for word in text.split():
#         if word not in stopwords.words('english'):
```

```

# for word in text.split():
#     if word in stopwords.words('english'):
#         new_text.append('')
#     else:
#         new_text.append(word)
# x=new_text[:]
# new_text.clear()
# return " ".join(x)

df=pd.read_csv("IMDB Dataset.csv")

# #lowercasing
# df['review']=df['review'].str.lower()
# #df.head()

# #remove html tags using regular expression
# df['review']=df['review'].apply(remove_htmltag)
# #df.head()

# #remove urls
# df['review']=df['review'].apply(remove_urls)
# #df.head()

# #remove punctuation
# df['review']=df['review'].apply(remove_punc)
# df.head()

# #chat word treatment
# #replace all the short form words like fyi,lamo etc with their autaul meanings

# #spelling correction
# df['review']=df['review'].apply(correct_spellings)
# #df.head()

# #removing stop words
# #words do not contribute to meaning of the sentence
# #required incase of sentiment analysis
# stop_words=stopwords.words('english')
# df['review']=df['review'].apply(remove_stopwords)
# df.head()

#handle emoji
#->1.remove
# df['review']=df['review'].apply(remove_emoji)
# df.head()
# 2.replace with its meaning
# df['review']=df['review'].str.emoji.demojize()
# df.head()

#tokenization
#it is important for text analysis
#word tokenising
#sentence tokenising

#methods
#1.using split function
# a="hello i am parth.I am new"
# b=a.split()
# c=a.split('.')
# print(b)
#2.using regular expression

#3.using nltk library
# from nltk.tokenize import word_tokenize,sent_tokenize
# a="hello i am parth.I am new"
# b=word_tokenize(a)
# c=sent_tokenize(a)
# print(b)
# print(c)

#4.using spacy lib for better results

#stemming
#bringing word to its root word

```

```

#use nltk porterstemmer

#lemmatization
#same as stemming , but the root word is always of part dictionary
#wordnet lemmitizer

```

Feature extraction from text

```

#to represent text in form of numbers preserving its semantic meaning
#method
#1.on hot encoding
#convert each words into (v)size vector
#it causes sparcity which leads to overfitting
#out of vocabuary problem
#does not capture semantic meaning

#2.bag of words
#usefull for text classification
#vector is formed by freq ,veotor size is vocabulary size

#use sklearn for implementation
# from sklearn.feature_extraction.text import CountVectorizer
# cv=CountVectorizer()
# X=cv.fit_transform(df['review']).toarray()
# print(X)

#creates sparcity
#little semantice meaning is captured
#word ordering is not considered

#3.n grams
#bigram->vocabulary is formed by comb of two words from the document
#trigram->vocabulary is formed by comb of three words from the document
#vector dimesnion increass if n increases,veotor size is vocabulary size

#4.tfidf
#veotor size is vocabulary size
#wieght based alg
#we maxiize the word weight if its freq in the curr doc is more , and it rarely occusre in the rest of the document
#term freq(tf)->freq of curr word/total words in the curr doc
#inverse document freq(idf)->log(total no of document/total no of documents in which curr word is present)+1
#weight(word)=tf*idf
#usefull for information reteival system(e.search engines)
#creates sparcity
#less semantic meaning is preserverd

#5.custom feautres
#6.word2vec(embdeiings)
#deep learning based methods
#tpyes ->1.cbow 2.skig gram
#less vector dim compared to other mehotds
#semantic meaning conserved
#eg if windows size is 3 ,text=i am parth ->here am =target and i ,parth =contxt wrds and its trained on the neural networjk
#for training words are one hot encoded

#important terms
#1.corpus ->concat of all text in the dataset
#2.document -> no of unqiue words in the corpus
#3.vocabulary ->each sample is the document
#4.word ->individual word of dcoument

```

```

#parts of speech tagging
# ->used in chat bots etc.
#question answering system
# import spacy
# nlp=spacy.load('en_core_web_sm')

```

```
# text="i am parth. i am from kolhpaur."  
# doc=nlp(text)  
# doc[0].pos_  
  
#pos algo  
#1.mark all the pos for the dataset  
#2.maintain a table for each word appearing as noun, verb etc.(this is emission probability)  
#maintain a table using start and end , calculate what occurs next to it(transmission probability)  
#for inference calculate all the possibilities, considering the output having high prob
```